TypeScript Task Tracker Dashboard with dynamic filtering and local storage persistence.

A Step-by-Step Project Guide

1. Project Overview

In this guide, you will build a modern, interactive Task Tracker Dashboard using HTML, CSS, and TypeScript. The application enables users to manage daily tasks with features like adding, toggling completion, deleting, dynamic status/priority filtering, and persistent storage using the browser's `localStorage` API. This project demonstrates core TypeScript concepts such as custom interfaces, Enums, type assertions, and DOM manipulation in a practical context.

2. Prerequisites

Before beginning this project, ensure you have the following installed and set up:

- **Node.js** (v16.0 or higher) and **npm** installed on your machine.
- A code editor such as **Visual Studio Code**.
- Basic familiarity with HTML5, CSS3, and modern JavaScript (ES6+).
- Fundamental knowledge of TypeScript concepts like types, interfaces, and function annotations.

3. Step-by-Step Instructions

Initialize the Project:

Open your terminal, create a new directory, and initialize a Vite project with TypeScript support:

```
mkdir task-tracker-ts
cd task-tracker-ts
npm create vite@latest . -- --template vanilla-ts
npm install
```

Project Directory Structure:

Organize your project files inside the `src` folder as follows:

```
task-tracker-ts/
■■■ index.html
■■■ src/
■   ■■■ style.css
■   ■■■ types.ts
■   ■■■ storage.ts
■   ■■■ main.ts
■■■ package.json
■■■ tsconfig.json
```

Define Types and Interfaces:

Create `src/types.ts` to hold your TypeScript types, such as task priority levels, filter states, and the core `Task` interface.

Implement Storage Persistence:

Create `src/storage.ts` to encapsulate `localStorage` reading and writing functions with proper TypeScript typing and error handling.

Build the User Interface:

Update `index.html` with a semantic form for adding tasks, control buttons for filtering, and a container for the task list.

Develop Core Application Logic:

In `src/main.ts`, handle state management, form submissions, dynamic dynamic rendering, filtering, and event delegation for task items.

Style the Dashboard:

Add CSS in `src/style.css` to ensure a modern, responsive layout with visual indicators for priorities and completion status.

Run the Development Server:

Start your application locally by running:

```
npm run dev
```

4. Code Examples

Defining Data Models (`src/types.ts`)

```
export type Priority = 'low' | 'medium' | 'high';
export type FilterStatus = 'all' | 'active' | 'completed';

export interface Task {
 id: string;
 title: string;
 priority: Priority;
 completed: boolean;
 createdAt: string;
}
```

LocalStorage Management (`src/storage.ts`)

```
import { Task } from './types';

const STORAGE_KEY = 'ts_task_tracker_tasks';

export const loadTasks = (): Task[] => {
 const data = localStorage.getItem(STORAGE_KEY);
 if (!data) return [];
 try {
   return JSON.parse(data) as Task[];
 } catch (error) {
   console.error('Failed to parse tasks from localStorage', error);
   return [];
 }
};
```

```
export const saveTasks = (tasks: Task[]): void => {
 localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
};
```

Main Application Logic (`src/main.ts`)

```
import './style.css';
import { Task, Priority, FilterStatus } from './types';
import { loadTasks, saveTasks } from './storage';

// State Management
let tasks: Task[] = loadTasks();
let currentFilter: FilterStatus = 'all';

// DOM Element Queries
const taskForm = document.querySelector<HTMLFormElement>('#task-form')!;
const taskInput = document.querySelector<HTMLInputElement>('#task-input')!;
const prioritySelect = document.querySelector<HTMLSelectElement>('#priority-select')!;
const taskList = document.querySelector<HTMLUListElement>('#task-list')!;
const filterButtons = document.querySelectorAll<HTMLButtonElement>('.filter-btn');

// Render Logic
const renderTasks = (): void => {
 taskList.innerHTML = '';

 const filteredTasks = tasks.filter(task => {
   if (currentFilter === 'active') return !task.completed;
   if (currentFilter === 'completed') return task.completed;
   return true;
 });

 filteredTasks.forEach(task => {
   const li = document.createElement('li');
   li.className = `task-item ${task.completed ? 'completed' : ''}`;
   li.innerHTML = `
     <div class="task-content">
       <input type="checkbox" ${task.completed ? 'checked' : ''} data-id="${task.id}" class="toggle-checkbox" />
       <span class="title">${escapeHtml(task.title)}</span>
       <span class="badge badge-${task.priority}">${task.priority}</span>
     </div>
     <button class="delete-btn" data-id="${task.id}">&times;</button>
   `;
   taskList.appendChild(li);
 });
};

const escapeHtml = (str: string): string => {
 const div = document.createElement('div');
 div.textContent = str;
 return div.innerHTML;
};

// Event Handlers
taskForm.addEventListener('submit', (e: SubmitEvent) => {
 e.preventDefault();
```

```
  const title = taskInput.value.trim();
  if (!title) return;

  const newTask: Task = {
    id: crypto.randomUUID(),
    title,
    priority: prioritySelect.value as Priority,
    completed: false,
    createdAt: new Date().toISOString()
  };

  tasks.push(newTask);
  saveTasks(tasks);
  renderTasks();
  taskInput.value = '';
});

taskList.addEventListener('click', (e: MouseEvent) => {
  const target = e.target as HTMLElement;

  if (target.classList.contains('toggle-checkbox')) {
    const id = target.getAttribute('data-id');
    tasks = tasks.map(t => t.id === id ? { ...t, completed: !t.completed } : t);
    saveTasks(tasks);
    renderTasks();
  }

  if (target.classList.contains('delete-btn')) {
    const id = target.getAttribute('data-id');
    tasks = tasks.filter(t => t.id !== id);
    saveTasks(tasks);
    renderTasks();
  }
});

filterButtons.forEach(btn => {
  btn.addEventListener('click', () => {
    filterButtons.forEach(b => b.classList.remove('active'));
    btn.classList.add('active');
    currentFilter = btn.dataset.filter as FilterStatus;
    renderTasks();
  });
});

// Initial Render
renderTasks();
```

5. Common Pitfalls

- **Null Type Errors during DOM Selection:** Forgetting that `document.querySelector` can return `null`. Use type assertions (`!` or `as HTMLElement`) only when you are certain the element exists in HTML.
- **Insecure HTML Injection (XSS):** Dynamically building DOM strings with raw user input using `innerHTML` opens up security vulnerabilities. Always sanitize user text or create elements using `document.createElement` and `textContent`.
- **Unsafe Type Casting:** Casting `localStorage` data using `as Task[]` without validation can cause runtime errors if stored data is corrupted. Always add fallback handling inside try-catch blocks.

- **Not Updating State and Storage Simultaneously:** Updating the UI array state without invoking `saveTasks` will result in lost changes upon browser refresh.

6. Expected Outcome

When completed, you will have a functional, single-page Task Tracker dashboard featuring:

- An input bar to add new tasks alongside a dropdown for priority level selection (Low, Medium, High).
- Filter control tabs (All, Active, Completed) that update the visual task list instantly without reloads.
- Color-coded visual badges indicating priority levels for each task.
- Checkboxes to toggle completion states and delete buttons to clear individual tasks.
- Persistent state retention when closing or refreshing the browser tab.

7. Next Steps

Enhance your task tracker dashboard further with these ideas:

- **Search & Sort:** Add a search input bar to filter tasks by title keyword and a sort dropdown by creation date or priority.
- **Editing Functionality:** Allow inline editing of existing task titles by double-clicking on task text.
- **Subtasks/Categories:** Extend the `Task` model to support tags or subtask completion arrays.
- **Dark Mode Toggle:** Implement a theme switcher utilizing CSS custom properties and persist preference in `localStorage`.
